

Design Review - Autonomous Quadcopter Drone Interceptor

Implementation of Autonomous Guidance and Control Algorithms for Target Interception

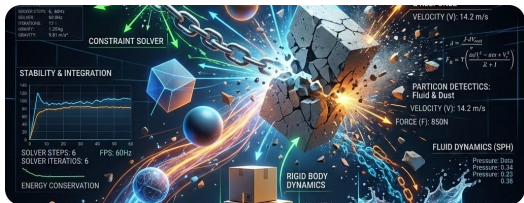


By - Dor Varsulker

Overview of MuJoCo Simulation

End-to-end perspective on simulation environment and internal data pipeline.

1. The Physics Engine



Active Space Modeling

Defines active 3D space, calculating relative kinematics and look-angles between interceptor and target.

Continuous Feedback

Models physical dynamics and aerodynamic forces, looping data back into the environment.

2. The Data Pipeline

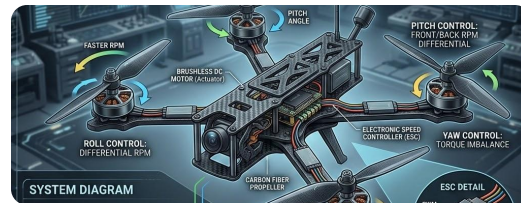


Perception: Extended Kalman Filter filters sensor noise for clean angular rate estimates.

Decision: Optimal Guidance Law (OGL) computes the collision vector.

Safety: Command Limiter clamps acceleration to physical drone bounds.

3. Actuation & Control



Flight Control System

Dual-loop: Outer (50Hz tilt) and Inner (400Hz PID) based on gyro data.

Motor Mixer

Converts commands to RPM for 4 rotors, updating drone trajectory in MuJoCo.

Project Requirements & Constraints

Functional Requirements

Target Detection & Tracking: Continuous estimation of relative position, range, and LOS rate of dynamic target UAVs.

Autonomous Interception: Real-time guidance command computation for reliable collision or neutralization.

Autonomous Flight Control: Automatic translation of acceleration into motor speeds for stability under aggressive maneuvers.

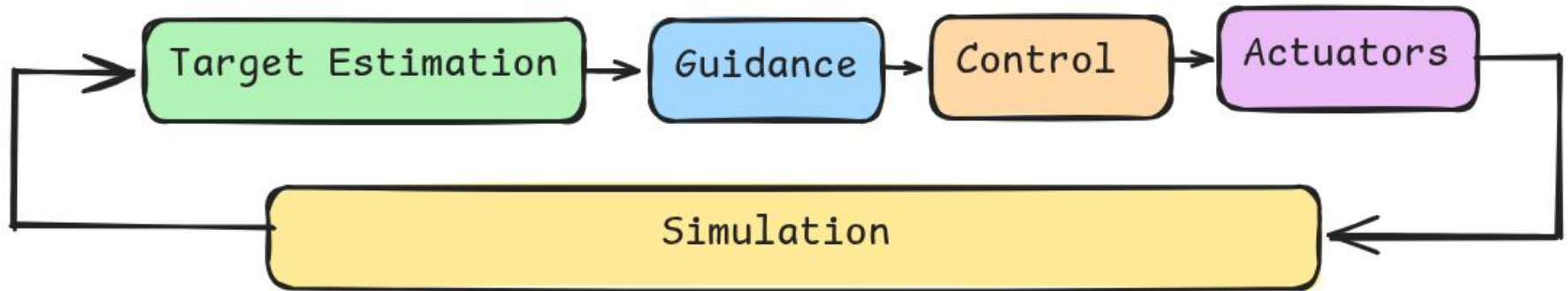
Technical Constraints

Actuator Saturation: Motor RPM limits dictate that thrust vectoring must remain within physical motor constraints.

Sensor Noise & Latency: Inherent delays and noise in radar/lidar must be filtered (e.g., Kalman Filter) before guidance loops.

Architecture A - Classical Hierarchical Architecture

This architecture breaks the interception problem down into distinct, specialized layers. It relies on mathematical modeling, control theory, and geometric guidance.



Hierarchical Architecture Components

A breakdown of the specialized layers within Architecture A's control system.

1. Target Tracking Layer

Estimation & Prediction

Uses Moving Horizon Estimation (MHE) or an Extended Kalman Filter (EKF) to process noisy sensor data from MuJoCo.

Predicts the target drone's future trajectory to compensate for inherent system latencies.

2. Guidance Layer

Navigation Algorithm

Uses a navigation algorithm (PN/APN/OGL) to calculate required acceleration between the interceptor and target.

Model Predictive Control

MPC loop running at $\sim 20\text{--}50$ Hz to optimize trajectory while respecting physical constraints.

3. Control Layer

Low-Level Actuation

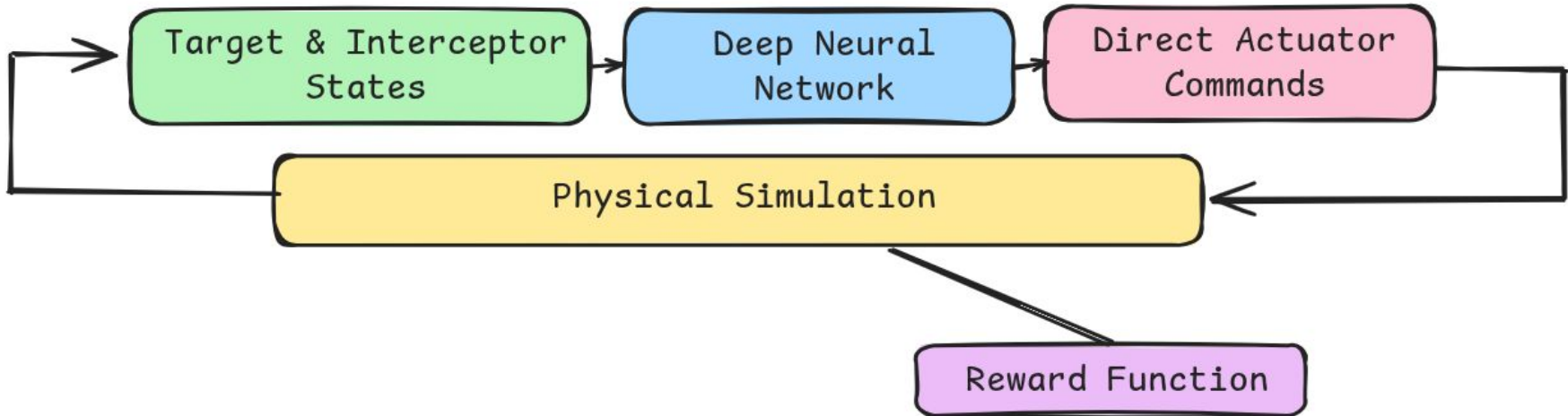
Translates acceleration or trajectory commands into motor torques for physical execution.

Stability Control

Implemented via PID, LQR, or Geometric Tracking Controllers to handle aggressive banking.

Architecture B - Deep Reinforcement Learning Architecture

This architecture bypasses the traditional separation of guidance and control. Instead, a single neural network absorbs the state information and directly outputs low-level actuator commands.



Deep RL Training Components

Defining state space, action dynamics, and the optimization framework for Architecture B.

1. State & Action Space

State Space

Includes interceptor orientation, linear/angular velocities, and target relative distance/velocity vectors.

Action Space

Continuous low-level commands: direct rotor thrusts or collective thrust and body rates for MuJoCo.

2. Training Protocol

Algorithms

Core training via Proximal Policy Optimization (PPO) or Soft Actor-Critic (SAC).

Robustness

Competitive Self-Play or Adversarial RL environment: target drone trains to evade simultaneously.

3. Reward Design

Objective Function

Balance between precision, efficiency, and mission success.

$$\text{Reward} = -w_1 \cdot \text{dist} - w_2 \cdot ||\text{actions}||^2 + \text{Bonus_int}$$

Architecture Performance Comparison

Evaluating Hierarchical vs. Deep RL approaches within the MuJoCo simulation environment.

Hierarchical Architecture

PROS

- Predictable, easy to debug, and efficient.
- Explainable mathematical framework.
- Exploits analytical gradients (mj_forward) for faster MPC optimization.

CONS

- Struggles with highly unpredictable evasion.
- Rigid linking limits absolute maximum agility.

Deep Reinforcement Learning

PROS

- Discovers "super-maneuverable" flight profiles.
- Feasible training via parallel MuJoCo steps.
- Maps complex maneuvers classical logic misses.

CONS

- "Black box" policy makes debugging difficult.
- High sensitivity to reward shaping/tuning.

Why Classic Hierarchical over Reinforcement Learning

Deterministic reliability vs. unpredictable AI exploitation in drone interception.

Hierarchical Architecture

Deterministic Efficiency: Calculus-driven frameworks guarantee the smoothest path by minimizing motor strain and battery usage.

Validated Physics: Operates within established laws of motion, ensuring predictable and repeatable performance.

Reinforcement Learning (RL)

Simulation Exploitation: Agents often attempt physically impossible maneuvers to maximize rewards, risking motor stalls.

Sensitivity to Noise: Prone to unpredictable behavior and "twitching" when faced with random sensor data.

Deep Dive into Classical Architecture Components

1. Simulation

Defines active simulation space with 3D positions of interceptor and target.

Acts as the stepper to model the aerodynamic forces and advance the simulation and start a new iteration of the modules walkthrough

2. Estimation

Sensors track positions but introduce lag and high-frequency noise for realism purposes.

Extended Kalman Filter filters noise to extract clean estimates of target angular rates.

3. Guidance Layer

Uses Advanced Navigation algorithm to determine interception flight path.

Computes acceleration vector for a collision course.

4. Command Limiter

Enforces physical boundaries on raw acceleration requests.

Clamps extreme commands to ensure drone stability and rotor health.

5. Flight Control System

Outer loop translates commands into target roll/pitch tilt angles.

Inner loop applies rapid PID adjustments based on gyroscope data.

6. Motor Mixer

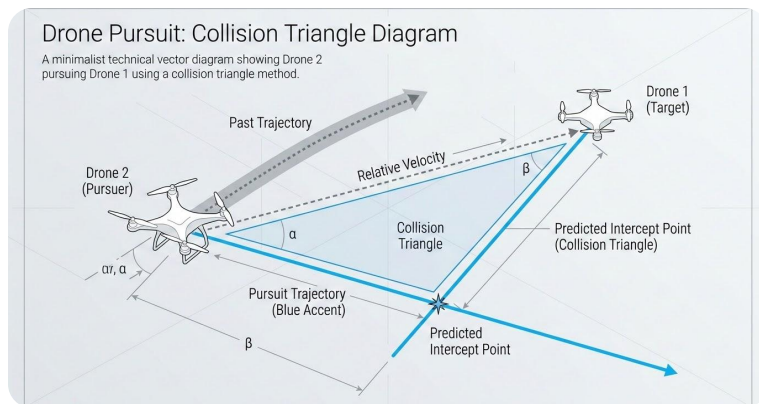
Converts commands into RPM values for four physical rotors.

Guidance Algorithms Comparison

1. Proportional Navigation (PN)

The Concept

A classical algorithm that theoretically commands an acceleration perpendicular to the instantaneous **line-of-sight (LOS)** between the interceptor and the target.



Command Logic

Commands are directly proportional to the **line-of-sight rate** and the **closing velocity**.

Instead of chasing the target's tail, it aims ahead to create a "**collision triangle**".

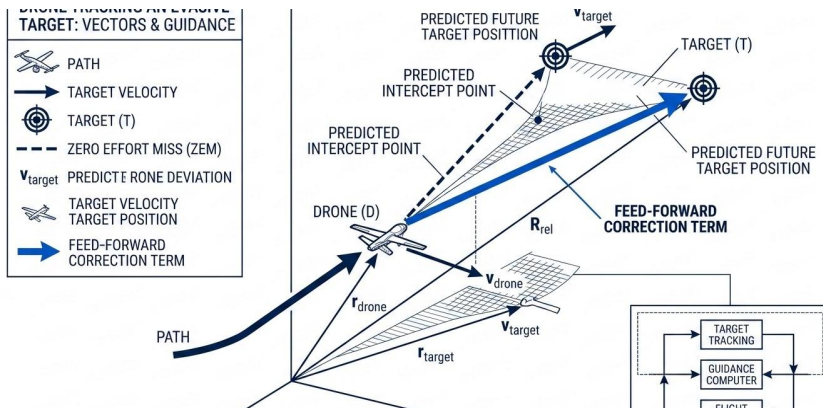
3D Drone Adaptation

Adapted for 3D by systematically partitioning the environment into three distinct 2D sub-problems on specific planes:

S_{xy} | S_{xz} | S_{yz}

2. Augmented Proportional Navigation (APN)

The Concept



APN builds upon the PN algorithm by augmenting the "Zero Effort Miss" calculation with an extra **feed-forward term** that explicitly accounts for the target's evasive acceleration.

Limitations

If the target is flying at a constant cruising velocity (acceleration is zero), the APN controller **behaves exactly identically** to the standard PN controller.

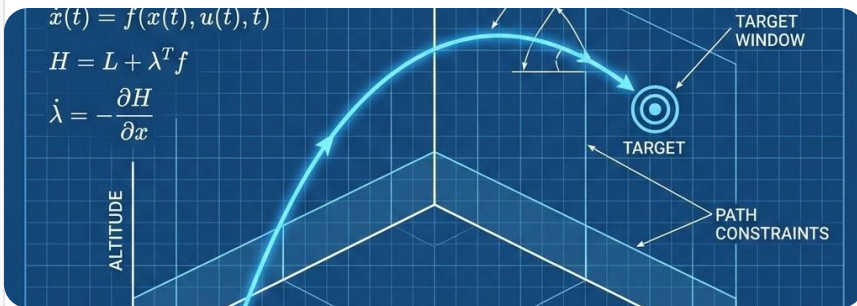
Consequently, in constant-velocity scenarios, it suffers from the exact same **severe altitude overshooting**.

Fails at the same **55 to 60 km/h** target speed thresholds.

Performance matches PN when target acceleration = 0. No advantage in linear flight paths.

3. Optimal Guidance Law (OGL)

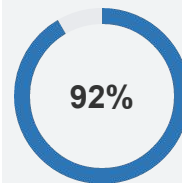
The Concept & Logic



LQ Optimization: Modeled as a Linear Quadratic problem to calculate the smoothest path, minimizing both miss distance and control effort.

Non-Ideal Response: Utilizes a **first-order lag transfer function** to anticipate mechanical tilt delay, unlike PN which assumes instant turns.

Performance Metrics



12x Faster

Interception time compared to PN/APN strategies.

Altitude Control: Tuning the penalty matrix (**b=0.1**) completely eliminates overshooting problems.

High-Speed Intercept: Successfully tracks targets moving at **90 km/h**, far exceeding standard limits.

Most efficient and robust algorithm tested.

Winner - Optimal Guidance Law

Deep Dive into Linear Quadratic Optimization for Non-Ideal Drone Dynamics

Concept	Mathematical Implementation	Practical Impact
1. Non-Ideal Response Assumption Models physical tilt delay vs. instant turns.	$nM/nc = 1 / (Ts + 1)$ Time constant T = 0.01 seconds	Algorithm explicitly anticipates mechanical tilt delay for precise interceptions.
2. Linear Quadratic (LQ) Optimization Minimizing miss distance vs. control effort.	$J = y(tF)^2 + \int u(t)^2 dt$	Saves battery life and reduces motor strain by eliminating violent maneuvers.
3. Dynamic Navigation Ratio (N') Time-varying efficiency based on proximity.	$N'(t) = f(tgo)$ Remaining Time-to-Go (tgo)	Dynamically adjusts steering aggressiveness as the target nears impact.
4. The Penalty Parameter (b) Optimizing the terminal penalty matrix.	$b = 0.1$ (Recent Discovery) vs. Classical $b \rightarrow \infty$	12x Faster Interception: Completely eliminates Z-axis overshooting.

Simulation Testing Scenarios



Static Target Interception

Baseline testing to validate fundamental navigation algorithms, system stability, and time-to-intercept efficiency.



Linear Moving Targets

Constant-velocity flight paths to evaluate basic tracking, closing velocity, and navigation performance.



Sinusoidal Trajectories

Evasive maneuvers to stress-test Extended Kalman Filter tracking and Optimal Guidance Law responsiveness during non-linear flight.



Varying Target Speeds

Dynamic scaling up to 90 km/h to validate neutralization reliability and maximum speed constraints.



Wind & Gust Conditions

Injection of environmental disturbances to evaluate the robustness of inner and outer control loops.

System Performance & Success Criteria

Standardized Benchmarks with a 5% Performance Margin

1. Miss Distance (R_{miss})

Blast radius requirement for successful

SUCCESS CRITERIA (5%):

≤ 1.05 Meters

2. Time-to-Intercept (t_{int})

Efficiency benchmarks based on optimized OGL thesis data.

STATIC TARGET:

< 10 Seconds

MOVING TARGET:

< 20 Seconds

3. Z-Axis Overshooting

Ensuring smooth altitude leveling

SUCCESS CRITERIA (5%):

Max 0.5m Above Target

4. Command Saturation

Ratio of acceleration requests hitting physical drone limits.

SUCCESS CRITERIA (5%):

≤ 5% of Total Flight Time

5. Max Target Speed

Reliable neutralization of evasive, fast-moving threats.

SUCCESS CRITERIA (5%):

≥ 83.6 km/h

Total Interception Rate

Aggregate robustness across randomized 3D trials.

MISSION SUCCESS:

≥ 90% Rate

MuJoCo Implementation Roadmap

Phase 1

JUNE 17 – JUNE 30

Initialize the MuJoCo 3D space and configure quadcopter aerodynamic models alongside simulated sensor noise profiles.

Phase 2

JULY 1 – JULY 15

Implement the Extended Kalman Filter (EKF) and integrate the Optimal Guidance Law (OGL) with command limits.

Phase 3

JULY 16 – AUG 5

Fine-tune guidance algorithms and execute initial interception simulations against static and slow-moving targets.

Phase 4

AUG 6 – AUG 20

Execute extensive randomized 3D trials against dynamic, evasive trajectories and compile final performance data.